

Introduction a REACT

C'est quoi React ?

- React est une bibliothèque JavaScript pour créer des interfaces utilisateur.

Concepts clés :

- Composants
- JSX
- props
- State (état)
- Événements

Qu'est-ce qu'un composant React ?

- ✓ Une application React est construite avec des **composants**.
- ✓ Un composant représente une **partie de l'interface utilisateur (UI)**.

Exemple :
Un bouton
Une carte produit
Une navbar
Une page complète

Chaque composant possède :
son apparence (UI)
sa logique (JavaScript)

Principe des composants

- React découpe l'interface en petits blocs réutilisables.
- ✓ Avantages :
 - Code réutilisable
 - Organisation claire
 - Maintenance facile

Un composant = une fonction JavaScript

- En React, un composant est souvent une **fonction JavaScript**.
- Elle retourne du **JSX** (du HTML dans du JS).

```
function Hello() {
  return <h1>Bonjour React</h1>;
}
```

Bonjour React

Le résultat de la fonction

Structure simple d'un composant

```
function Button() {
  return (
    <button>Cliquez ici</button>
  );
}
```

function Button() → définition du composant
return → ce qui sera affiché
JSX → mélange HTML + JavaScript

Utiliser un composant

- Un composant peut être utilisé comme une balise HTML :

```
function Button() {
  return (
    <button>Cliquez ici</button>
  );
}

function App() {
  return (
    <div>
      <h1>Bonjour React</h1>
      <Button />
    </div>
  );
}
```

Bonjour React

Cliquez ici

Le résultat de la fonction

Important :
 Le nom du composant commence par une majuscule.

Exemple concret d'une app

```
function Navbar() {
  return <h2>Menu</h2>;
}

function Content() {
  return <p>Contenu principal</p>;
}

function Button() {
  return (
    <button>Cliquez ici</button>
  );
}

function App() {
  return (
    <>
      <h1>Bonjour React</h1>
      <Button />
      <Navbar />
      <Content />
    </>
  );
}

export default App;
```

L'application est composée de plusieurs composants.

Export default et export

- on utilise **export** pour rendre un élément accessible depuis un autre fichier.
- Il existe **2 types** :
- export nommé → export
- export par défaut → export default

Export default

Les mots-clés **export default** permettent de définir le **composant principal** d'un fichier.

Permet d'utiliser le composant ailleurs
Rend le code modulaire
Facilite l'organisation des projets React

export nommé (export simple)

Principe

- Permet d'exporter **plusieurs éléments** depuis un même fichier.
- Exemple:

fichier : math.js

```
export const PI = 3.14;
export function addition(a, b) {
  return a + b;
}

export function soustraction(a, b) {
  return a - b;
}
```

Import

fichier : main.js

```
import { PI, addition, soustraction }
from "./math.js";

console.log(PI);
console.log(addition(2,3));
```

On utilise :

```
{}
```

export default

• Principe:

Permet d'exporter **un seul élément principal**

Exemple
fichier : Person.js

```
export default function Person() {
  console.log("Bonjour");
}
```

Import

```
import Person from "./Person.js";

Person();
```

Importation sans:

```
{}
```

Peut-on utiliser les deux

```
export const age = 20;
export default function User() {}
```

```
import
```

```
import User, { age } from "./User";
```

Ajouter des styles en React

Ajouter des styles en React

- En React, pour appliquer une **classe CSS**, on utilise **className** (et non **class** comme en HTML).

```
<img className="avatar" />
```

On écrit le CSS dans un fichier séparé, comme en HTML.

App.css

```
.avatar {
  border-radius: 50%;
  width: 100px;
  height: 100px;
}
```

Lier le CSS dans React

```
import './App.css';

function App() {
  return ;
}

export default App;
```

Les props

Les props

- **Props** = *properties* (propriétés)
- Elles permettent de **transmettre des données d'un composant parent vers un composant enfant**.
- Ce sont **des valeurs en lecture seule**, donc le composant enfant **ne peut pas les modifier**.

Exemple

```
import React from "react";

function Enfant(props) {
  return <h1>Bonjour, {props.nom} !</h1>;
}

function App() {
  return (
    <div>
      <Enfant nom="Salma" />
      <Enfant nom="Ali" />
    </div>
  );
}

export default App;
```

- ✓ App est le composant parent.
- ✓ Enfant est le composant enfant.
- ✓ On passe nom="Salma" comme **prop** au composant enfant.
- ✓ Dans Enfant, on récupère cette valeur avec props.nom.

Déstructuration des props

```
import React from "react";

function Enfant(props) {
  return <h1>Bonjour, {props.nom} !</h1>;
}

function App() {
  return (
    <div>
      <Enfant nom="Salma" />
      <Enfant nom="Ali" />
    </div>
  );
}

export default App;
```

Au lieu de faire props.nom, on peut directement **déstructurer** :

```
function Enfant({ nom }) {
  return <h1>Bonjour, {nom} !</h1>;
}
```

Props multiples

```
function Profil({ nom, age }) {
  return (
    <p>
      {nom} a {age} ans.
    </p>
  );
}

function App() {
  return <Profil nom=" Mohamed" age={25} />;
}
```

- ✓ On peut passer plusieurs props.
- ✓ Les props peuvent être de **n'importe quel type** : string, number, boolean, tableau, objet, fonction

State : le cœur de React

useState est un **Hook** qui permet de :

- ✓ stocker une valeur
- ✓ modifier la valeur
- ✓ mettre à jour l'affichage automatiquement

Pourquoi useState ?

- En JavaScript normal :

```
let x = 0;
x = x + 1;
```

L'écran ne se met PAS à jour automatiquement.

Mais avec React : `useState`

React met à jour l'écran automatiquement.

Syntaxe useState

```
const [variable, setVariable] = useState(valeurInitiale);
```

Exemple :

```
const [age, setAge] = useState(20);
```

élément	rôle
age	valeur actuelle
setAge	fonction pour modifier
20	valeur initiale

Exemple 1 : Compteur simple

```
import { useState } from "react";

function Compteur(){
  const [nombre, setNombre] = useState(0);
  return(
    <div>
      <h1>{nombre}</h1>
      <button onClick={() => setNombre(nombre + 1)}>
        Ajouter
      </button>
    </div>);
}

export default Compteur;
```

Au début :

nombre = 0

Click :

setNombre(nombre + 1)

Devient :

nombre = 1

React met à jour l'écran

State : le cœur de React

State : le cœur de React

```
import { useState } from "react";

function App() {
  const [message, setMessage] = useState("Bonjour");

  return (
    <div>
      <h1>{message}</h1>
      <button onClick={() => setMessage("Bienvenue")}>
        Changer le message
      </button>
    </div>
  );
}

export default App;
```

Remarque :

- ✗ Pas de getElementById
- ✗ Pas de textContent
- ✓ React met à jour le DOM tout seul

Les événements en React

Les événements en React

- Les **événements (Events)** permettent de réagir aux actions de l'utilisateur :
- clic
- écriture
- survol
- soumission formulaire

Syntaxe générale

```
<button onClick={fonction}>
Click
</button>
```

Exemple des événements

Événement	Description	Exemple React
onClick	Déclenché lorsqu'on clique sur un élément	button onClick={() => alert("Clic !")}>Cliquez-moi
onDoubleClick	Déclenché lorsqu'on double-clique sur un élément	div onDoubleClick={() => alert("Double clic !")}>Double-cliquez ici
onChange	Déclenché lorsqu'une valeur d'input ou textarea change	input onChange={(e) => console.log(e.target.value)} />
onSubmit	Déclenché lors de la soumission d'un formulaire	form onSubmit={(e) => { e.preventDefault(); alert("Envoyé !"); }}>Envoyer
onKeyDown	Déclenché lorsqu'une touche du clavier est pressée	input onKeyDown={(e) => console.log("Touche :", e.key)} />
onMouseOver	Déclenché lorsque la souris passe au-dessus d'un élément	h1 onMouseOver={() => console.log("Souris dessus")}>Survolez-moi
onMouseLeave	Déclenché lorsque la souris quitte un élément	div onMouseLeave={() => console.log("Souris partie")}>Souris ici
onFocus	Déclenché lorsqu'un élément reçoit le focus (clic ou tab)	input onFocus={() => console.log("Focus")} />
onBlur	Déclenché lorsqu'un élément perd le focus	input onBlur={() => console.log("Perte du focus")} />

Exemple

```
function Test(){
  function afficher(){
    alert("Bonjour");
  }
  return(
    <button onClick={afficher}>
      Click
    </button>
  );
}

export default Test;
```

Exemple (onChange (Input))

```
import { useState } from "react";

function Formulaire(){
  const [nom, setNom] = useState("");

  function changer(e){
    setNom(e.target.value);
  }

  return(
    <div>
      <input onChange={changer} />
      <h1>{nom}</h1>
    </div> );
}
```

e.target.value
Permet de récupérer la valeur